

RMDB 数据库系统设计与技术实现文档

赛事：2026 全国大学生计算机系统能力大赛 · 数据库管理系统设计赛

文档用途：说明本项目在官方 RMDB 框架基础上的系统架构、核心模块设计、关键实现机制、性能优化方法与测试验证体系。

代码基线：fe73e7c（分支 feat/perf-fixes）

文档约定

- 本文档以代码基线 fe73e7c 为主要描述对象；第 12.1 节的官方性能指标来自文中单独标注的评测提交 main @ 0e544e4。若不同提交之间存在实现差异，以对应章节明确标注的提交版本为准。
- 文中标注为“实测”的数据均来源于项目测试或评测记录；测试条件和可比性约束见第 13.3 节。明确标注为“官方评测”的数据仅用于描述相应官方评测结果。
- 代码路径、类名、函数名、协议字段及常量均使用反引号标记；英文缩写首次出现时给出中文语义或上下文说明。
- 本文档重点描述当前实现及其设计边界，不将未实现能力表述为系统已具备功能。

目录

- 项目概述
- 整体架构
- 网络层与 Wire Protocol v3
- SQL 前端：解析、语义分析、查询优化
- 执行引擎
- 存储引擎
- 索引：B+ 树
- 事务与并发控制
- 日志与故障恢复
- 关键设计决策与权衡
- 关键实现不变量与风险约束
- 性能分析与优化结果
- 测试与验证体系
- 已知限制与后续方向

1. 项目概述

1.1 系统定位

RMDB 是中国人民大学数据库教学团队为本赛道提供的代码框架。本项目基于该框架实现面向赛题功能与性能要求的单机关系型数据库管理系统，主要使用 C++17 开发。当前系统具备：

- 端到端 SQL 处理链路：词法/语法分析 → 语义分析 → 查询优化 → 火山模型执行
- 基于缓冲池的页式存储引擎，以及堆文件与 B+ 树索引
- 多版本并发控制（MVCC），支持快照隔离（SI）与可串行化快照隔离（SSI）
- 基于预写日志（WAL）与静态检查点的故障恢复机制
- 按决赛规范实现的 RMDB Wire Protocol v3 二进制通信协议

决赛负载为 TPC-C，规模 50 仓库、约 2,500 万行、库大小约 4.45 GiB，32 客户端并发。

1.2 项目规模

项	数量
手写源码（.cpp + .h，不含 flex/bison 生成物）	18,522 行
含生成物	24,516 行
执行算子	16 个
SQL 回归用例	28 个
专项验证脚本（gates）	19 个
有记录的优化/修复轮次	R001 ~ R043

1.3 功能范围与赛题映射

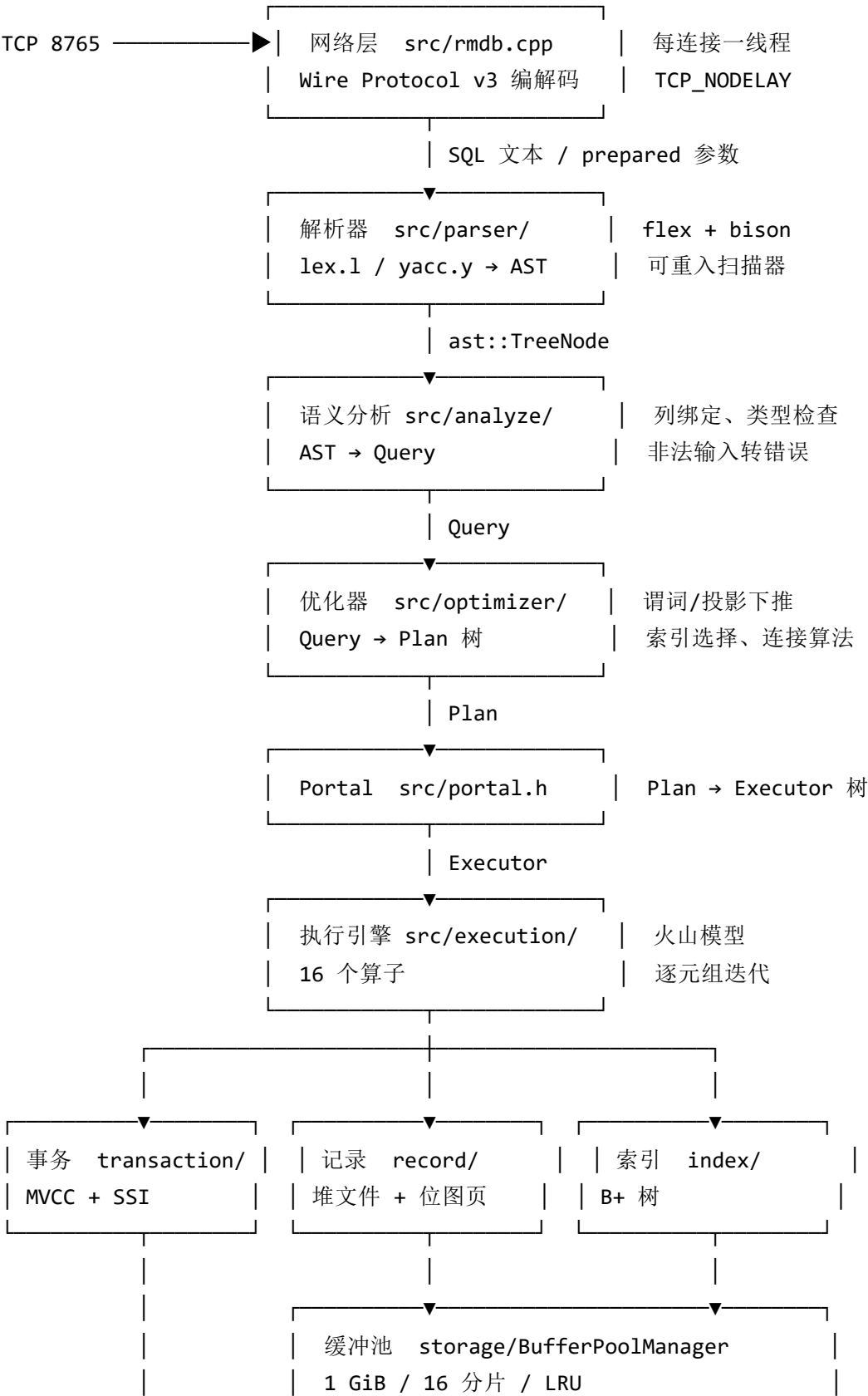
题号	内容	主要落点
一	存储引擎	src/storage/、src/record/、src/replacer/
二	查询执行基础	src/parser/、src/analyze/、src/optimizer/、src/execution/
三	唯一索引	src/index/
四	EXPLAIN ANALYZE	src/execution/explain_printer.h

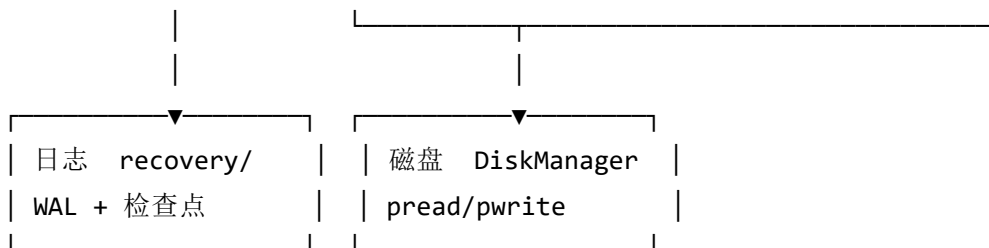
题号	内容	主要落点
五	聚合与分组	src/execution/executor_aggregate.h
六	UNION 集合算子	src/execution/executor_union.h
七	连接优化（NLJ / INLJ）	src/execution/executor_{nestedloop,index,hash}_join.h
八	事务控制	src/transaction/transaction_manager.cpp
九	隔离级别（SI / SSI）	同上 + watermark.cpp
十	WAL 故障恢复	src/recovery/

决赛阶段在上述功能基础上进一步覆盖 Wire Protocol v3、COUNT(DISTINCT)、FLOAT32 位级精度、COMMIT 持久化审计、崩溃后一致性校验以及 TPC-C 性能评测。

2. 整体架构

2.1 分层视图





2.2 SQL 请求的端到端处理路径

以 `select sum(ol_amount) from order_line where ol_w_id=1 and ol_d_id=2;` 为例：

1. **网络层**（`rmdb.cpp:client_handler`）读入 8 字节帧头 + payload，识别 `EXEC_STREAM (0x20)`，取出 SQL 文本。
2. **解析**（`parse_one_sql`）用可重入 flex 扫描器 + bison 生成 `ast::SelectStmt`。每次解析持有独立 `yyscan_t`，无全局词法状态，多连接可并行解析。
3. **语义分析**（`Analyze::do_analyze`）把 `ol_w_id` 绑定到 `order_line.ol_w_id`，校验类型，产出 `Query{sel_cols, tabs, conds, group_by, having, limit}`。
4. **优化**（`Planner::plan_query`）：
 - `get_index_cols` 按最左前缀匹配，选中索引（`ol_w_id, ol_d_id, ol_o_id, ol_number`），前缀命中 2 列
 - 生成 `ScanPlan{T_IndexScan}`，残余条件交由算子内部过滤
 - 外层套 `AggregatePlan` → `ProjectionPlan`
5. **Portal** 递归把 Plan 树转成 Executor 树。
6. **执行**：`ProjectionExecutor::beginTuple()` → `AggregateExecutor::beginTuple()` → `IndexScanExecutor` 沿 B+ 树叶链遍历区间，逐行经 MVCC 可见性判定后交给聚合器。
7. **回写**：结果经 `wire::ResultWriter` 编码为 `META` → `ROW*` → `RESULT_END` 三段帧。

2.3 并发模型

- **连接**：`accept()` 后 `pthread_create` 一线程一连接，无线程池。
- **单连接串行**：协议规定同一连接任意时刻只有一个 outstanding request，不做 pipeline。
- **并行度来源**：多连接并发。数据结构按分片解耦（缓冲池 16 片、版本存储 64 片），访问不同页或不同行的数据路径可并行执行。

3. 网络层与 Wire Protocol v3

3.1 协议设计动机

框架自带的 `rmdb_client` 采用行式文本协议：每条 SQL 以文本发送，结果同样以文本形式返回。在决赛事务负载中，单笔事务需要发送 5~40 条语句，该模式会引入重复解析、SQL 文本传输以及逐条 DML 确认等额外开销。为满足决赛协议与性能要求，系统实现了官方定义的二进制 Wire Protocol v3。

3.2 帧格式

握手阶段由客户端发送 8 字节 `RMDB\x00\x03\x00\x00`，服务端校验后原样回送；若握手内容不匹配，则终止该连接。握手数据必须在协议层完成消费，不能进入 SQL 解析器。

之后所有请求/响应共用 8 字节帧头（大端）：

```
u32 payload_bytes    // 不含本帧头
u8  tag
u8  flags
u16 reserved         // 必须为 0
u8  payload[payload_bytes]
```

单帧 payload 上限 1 MiB，诊断文本上限 64 KiB。

方向	tag	名称	用途
C→S	0x20	EXEC_STREAM	执行一条普通 SQL，结果流式返回
C→S	0x21	PREPARE_SET	安装本连接的语句字典（≤256 条）
C→S	0x22	EXEC_BATCH	顺序执行一组已准备操作（ <code>flags</code> 必须为 <code>0x01</code> ，即 <code>AUTO_ABORT</code> ）
S→C	0x01	META	查询列定义
S→C	0x02	ROW	一行类型化数据
S→C	0x10	COMMAND_OK	非查询成功
S→C	0x11	RESULT_END	查询成功终结，携带 u64 <code>row_count</code>
S→C	0x12	TRANSACTION_ABORT	事务已被服务端回滚

方向	tag	名称	用途
S→C	0x13	ERROR	请求失败
S→C	0x14	PREPARE_OK	字典安装成功 + 查询 schema
S→C	0x15	BATCH_RESULT	EXEC_BATCH 的唯一压缩响应

3.3 请求执行路径

EXEC_STREAM 面向任意 SQL 与大结果：META → ROW* → RESULT_END 流式发送。

功能测试、恢复、建表、装载与一致性查询均使用该路径。

PREPARE_SET + EXEC_BATCH 面向性能高频路径。每连接只准备一次 SQL 模板、参数类型与结果模式（schema），后续批处理仅发送 statement id 与类型化参数值，响应只携带查询行。TPC-C 五类事务在批处理路径下的往返次数：NewOrder 2、Payment 2、OrderStatus 3、Delivery 2~3、StockLevel 2；按 45/43/4/4/4 混合比例加权约 2.08 次 RTT/事务。

3.4 关键实现约束

- **循环读写**：read_exact / write_all 必须循环直到完整字段传输完毕，不能假定一次 recv / send 得到完整帧。
- **恰好一个终结帧**：每个请求必须产生且仅产生一个终结帧。execute_stream_sql 的正常路径与四条 catch 路径各自恰好发一帧；ResultWriter::terminal_diagnostic 发送终结帧后调用 clear_result()，保证「已发 META+ROW 再出错」也只追加一个 ERROR。
- **AUTO_ABORT 原子性约束**：批处理内任一操作失败时，若连接上存在活动事务，服务端必须在发失败响应之前完成回滚；失败响应不得携带任何部分查询结果。
- **类型化传输**：FLOAT 按 IEEE-754 binary32 位模式直接传输，不经十进制文本转换；CHAR 发逻辑字符串原始字节，不含页内定长 padding、不追加 NUL。

3.5 静态检查点命令的专用分发路径

create static_checkpoint；不走常规 SQL 路径。is_static_checkpoint_cmd() 在分发前做大小写/空白容忍的字面匹配，命中后直接调 create_static_checkpoint()。该设计将运维控制命令与常规 SQL 语法解析解耦，避免仅为单一管理命令扩展通用 SQL 语法。

4. SQL 前端：解析、语义分析、查询优化

4.1 解析器

lex.l (flex) + yacc.y (bison) → ast.h 中 TreeNode 派生的节点树。

关键设计：

- **可重入扫描器**。%option reentrant bison-bridge bison-locations，每次解析持有独立 yyscan_t。改造前全局 flex 缓冲是串行化根因，需要在 rmdb.cpp 里用一把 buffer_mutex 包住整段 scan + parse + analyze；改造后该锁被移除，多连接批量装载场景下的解析与语义分析可以并发执行。
- **LOAD 路径的独占起始状态**。文件路径含 / . - 等普通规则识别不了的字符，用 %x STATE_LOAD_PATH 单独处理：跳过前导空白，捕获到下一个空白/分号为止。模式是 [^\t\r\n;]+，因此裸文件名、./、../、绝对路径都能解析。
- **未知字符返回错误码而非吞掉**。兜底 . 规则返回 YYUNDEF 触发语法错误。早期实现仅输出诊断信息而未返回错误 token，可能使 != 被错误解析为 =，从而改变谓词语义并导致查询结果错误。

维护约束：语法修改应同步更新 lex.l 与 yacc.y，随后重新生成解析器代码；

lex.yy.cpp 与 yacc.tab.cpp 属于生成文件，不应直接修改。

4.2 语义分析

Analyze::do_analyze 把 AST 转成类型化的 Query：

- **列名消歧**：TabCol{tab_name, col_name} 绑定。无表名前缀时在 FROM 涉及的表中查找，重名报 AmbiguousColumnError。
- **类型检查与提升**：INT → FLOAT 隐式提升；CHAR(n) 与 CHAR(m) 比较取 max。
- **非法输入转错误不崩溃**：未知列、类型不兼容、聚合嵌套等统一抛出 RMDBError，由上层转成 ERROR 帧。

4.3 优化器

planner.cpp 将 Query 编译为 Plan 树，当前主要包含以下五类优化：

(1) 谓词下推与索引选择

get_index_cols(tab_name, disp, conds, out_index_cols) 按**最左前缀**在该表所有索引里挑能匹配最长前缀的：

```
对每个索引 idx:
    match = 0
    对 idx 的每一列 col:
        kind = cond_kind(col)           // 0=无谓词, 1=范围, 2=等值
        if kind == 0: break
        match++
        if kind == 1: break             // 范围列之后不再向后匹配
    取 match 最大的索引
```

命中则生成 T_IndexScan，残余条件由 IndexScanExecutor 内部过滤，不再套 Filter；
未命中则 T_SeqScan + 上层 FilterPlan。

隔离级别相关：force_seqscan_iso() 使 SERIALIZABLE 恒走 SeqScan。SSI 的读集与谓词登记粒度需要保持既定验证基线的表粒度语义；IndexScan 只命中当前物理索引键，并发改键会让旧快照经索引漏行、依赖图变形。SNAPSHOT（性能测试规定的隔离级）则放开 IndexScan——快照隔离本就允许幻读，且 TPC-C 索引建在主键上而事务只改非键列。

(2) 连接算法选择

左深树。以 tables[0] 起步，后续每步优先挑「与已连接集合存在连接条件」的表，避免中间笛卡尔积（FROM A,B,C WHERE A.x=C.x AND B.y=C.y 会按 A→C→B 连接）。

对每一步的内表按优先级选算法：

优先级	算法	触发条件
1	INLJ (索引连接)	try_upgrade_inlj 能用「内表固定等值谓词 + 连接列绑定到外表列」凑出索引最左 EQ 前缀
2	哈希连接	存在跨表等值条件，且内表规模超过阈值
3	朴素嵌套循环	以上都不成立（小表连接保持既有形态与 EXPLAIN 计数）

INLJ 升级时改写 ScanPlan::tag = T_IndexScan、填 probe_bindings_（索引列下标 → 外侧 TabCol）与 probe_prefix_len_，Portal 据此构造 IndexJoinExecutor。

(3) 投影下推

project_pushdown 计算每个子树需要向上暴露的列集合，只在跨过 Join 时插入下推的 Projection——单表查询不需要内层 Projection。

(4) 索引跳跃扫描 (skip scan)

该优化是当前记录中对整体性能提升最显著的单项优化。

问题形状：谓词绑住了索引第 2..k 列却没绑首列。TPC-C Delivery 的

```
select sum(ol_amount) from order_line where ol_d_id=? and ol_o_id=?
```

索引是 (ol_w_id, ol_d_id, ol_o_id, ol_number)，ol_w_id 未绑定，最左前缀匹配不上，计划会退化为约 1,500 万行的全范围扫描。实测该语句耗时 1395.6 ms；作为定位瓶颈的对照实验，补充 ol_w_id 条件后耗时为 0.115 ms。

做法：把首列的**不同取值**从索引本身枚举出来，每个取值做一次正常的等值前缀区间扫描。首列基数低时（ol_w_id 只有 50 个仓库）总代价是 D 次索引下降而非 N 行。思路对应常见数据库系统中的 index skip scan 类优化思路。

边界处理：

- 只支持恰好一列未绑定且该列为 INT。后继值就是 v+1，无需按类型推导后继，也不必处理多列进位。多列/字符串跳跃留给后续轮次。
- 探测次数保护阈值 kMaxSkipProbes = 4096。当首列基数较高时，逐值进行索引下降的成本可能超过顺序扫描；达到阈值后降级为从当前位置扫描至索引末尾。两条路径均访问候选键的超集，并由 fed_conds_ 完成最终条件过滤，以保持结果语义一致。
- 判定权归 planner (ScanPlan::use_skip_scan_)。执行器只看得到 conds_，而 INLJ 内表的索引首列是被**连接条件**绑定的 (is_rhs_val=false，运行时才由外行喂值)，从条件里看去与「首列完全无谓词」无法区分。自行推断会把内表点查变成扫遍首列全部取值的跳跃扫描。

(5) MIN 沿索引序下推

SELECT MIN(c) FROM t WHERE k1=v1 AND ... AND kp=vp，若所选索引恰好是 (k1..kp, c, ...)，扫描区间在 c 上天然升序，**第一行就是 MIN**，无需扫完整个区间。对应 TPC-C Delivery 的 select min(no_o_id) from new_orders where no_w_id=? and no_d_id=?。思路同 PostgreSQL 的 preprocess_minmax_aggregates。

当前仅对 MIN 启用该优化。MAX 需要反向扫描，而 IxScan 现有实现沿叶链正向推进，且叶链为环形结构（见 §11）；若支持反向遍历，需要重新定义并验证终止条件，因此暂未纳入当前实现。

INLJ 内表不适用——内表由外行驱动、多次重置，第一行不是全局 MIN。

5. 执行引擎

5.1 火山模型

所有算子继承 AbstractExecutor ，实现同一组接口：

```
virtual void beginTuple();           // 定位到第一条结果
virtual void nextTuple();           // 推进到下一条
virtual std::unique_ptr<RmRecord> Next(); // 取出当前元组
virtual bool is_end() const;
virtual const std::vector<ColMeta>& cols() const;
virtual size_t tupleLen() const;
```

算子实现全部是头文件类（executor_*.h），Portal 递归把 Plan 树转成 Executor 树。
rows_out_ 计数供 EXPLAIN ANALYZE 使用。

5.2 算子清单

类别	算子	说明
扫描	SeqScanExecutor	全表扫描，跳过位图未置位的槽
	IndexScanExecutor	索引区间扫描 + skip scan + MIN 下推
连接	NestedLoopJoinExecutor	朴素嵌套循环
	IndexJoinExecutor	索引连接（内表按外行探测）
	HashJoinExecutor	哈希连接
修改	InsertExecutor / UpdateExecutor / DeleteExecutor	含 MVCC 写路径与索引维护

类别	算子	说明
	LoadExecutor	CSV 批量装载
计算	AggregateExecutor	分组、聚合、HAVING
	ProjectionExecutor / FilterExecutor	投影 / 过滤
	SortExecutor / LimitExecutor	排序 / 限流
	UnionExecutor	集合并与去重

5.3 扫描算子的 MVCC 集成

扫描算子不直接读堆，而是经 `mvcc_visible_record()`：

```
// execution_common.h
if (非 MVCC 隔离级) return fh->get_record(rid, context);
auto vis = tm->VisibleRecordForScan(tab_name, rid, txn); // 只读扫描专用
```

`VisibleRecordForScan` 与 `VisibleRecord` 同语义，但**内存无版本时直接读堆返回，不补种、不插版本表**。原因：

- 若全表扫描逐行补种版本，会使整张表的 `RowVersion` 重新进入内存，产生 $O(\text{表规模})$ 的额外驻留并抵消 GC 效果
- 每行还要付 `EnsureSeededFromHeap` 的写锁 + 插表代价，扫描随规模超线性变慢

正确性论证：无内存版本 $\Rightarrow$ 最新已提交版本要么被 GC (`commit_ts < watermark`)、要么是恢复基准版本 $\Rightarrow$ 对任何当前读者都可见 (`start_ts \geq watermark > commit_ts`)，堆里正是该最新已提交值。已删行的墓碑被 GC 排除而保留在链上，走 `chain` 分支返回 `nullptr`，不会落到读堆路径被误复活。写路径仍用 `VisibleRecord`（照常补种）。

5.4 聚合算子

- **分组按插入序**，不是 `std::map` 字典序——题面要求输出顺序与首次出现顺序一致。用哈希表定位分组 + `vector` 保持顺序。
- **FLOAT 累加用 double**：SUM / AVG 内部以 `double` 累加，最后一次性舍入回 `float` 输出。这正是官方 FLOAT 规则第 3 条：「把每个已存在的 `binary32` 输入精确扩展为 `binary64`，在 `binary64` 中累加，最后只舍入一次为 `binary32`」。
- **COUNT(DISTINCT col)**：为去重构造精确键，INT/CHAR 沿用存储字节。语法上同时兼容 `COUNT(DISTINCT col)` 与 `COUNT(DISTINCT (col))`。

- **HAVING 侧信道累加器**：HAVING 里引用的聚合可能不在 SELECT 列表中，需要独立维护一份累加器。

5.5 UNION

多分支物化 + 类型提升（INT→FLOAT、CHAR(n)/CHAR(m)→CHAR(max)）+ 整行二进制去重。

6. 存储引擎

6.1 磁盘管理

DiskManager 按 (fd, page_no) 做 pread / pwrite，维护 fd2path_ / path2fd_ 双向映射。日志文件单独用 write_log（追加）+ sync_log（fdatasync）。

6.2 缓冲池

BufferPoolManager 是分片式页缓存：

参数	值
页大小	4 KiB
池容量	262,144 页 = 1 GiB
分片数	16
淘汰策略	LRU（每分片一个 LRUReplacer）

分片一致性是关键不变量：frame 按 frame_id % num_shards_ 静态归属，page 按 hash(page_id) % num_shards_ 归属，两者一致，因此被淘汰的旧页与新页必属同一分片，装入/淘汰全程零跨片。

```
Page* BufferPoolManager::fetch_page(PageId page_id) {
    Shard& shard = shard_for(page_id);
    std::scoped_lock lock{shard.latch};
    // 命中 → pin++; 未命中 → find_victim_page → update_page → read_page
}
```

两点需要注意：

- `fetch_page` 在**池耗尽且无可淘汰**时返回 `nullptr`，不进行阻塞等待或自旋。部分调用方尚未检查该返回值，这是当前实现需要进一步加固的边界。
- `update_page` 淘汰脏页时会先调 `flush_log_cb_()`（WAL 先行规则：脏页落盘前相关日志必须先落盘），这一步在持有分片 latch 的情况下进行。

6.3 记录管理（堆文件）

`RmFileHandle` 管理一张表的堆文件：

```
page 0      : RmFileHdr { record_size, num_pages, num_records_per_page,
                        first_free_page_no, bitmap_size }
page 1..N-1 : RmPageHdr { next_free_page_no, num_records }
              + bitmap（每记录 1 bit）
              + 定长记录槽 × num_records_per_page
```

- **空闲空间管理**：文件头维护 `first_free_page_no`，非满页以 `next_free_page_no` 串成单向链表。插入 $O(1)$ 摘链头，页满则摘除。
- **插入路径持** `heap_latch_`（每表一把互斥锁），覆盖 INSERT 与 DELETE。
- **非法 RID 防护**：`insert_record` 不返回 `Rid{-1,-1}`。若非法 RID 被上层写入 WAL，会导致后续恢复无法按有效物理位置重放，并可能造成记录丢失。若空闲链头指向满页，则将该页摘链并换页重试。

6.4 扫描游标

`RmScan` 逐槽遍历，跨页时复用已 pin 的帧（`fetch_page_pinned` 命中同页直接返回），保证 pin 严格守恒——任何离开当前页/扫描结束/析构都经 `release_cur_page()`。

7. 索引：B+ 树

7.1 结构

`IxIndexHandle` 每个索引一棵 B+ 树，页布局：

```
page 0 : IxFileHdr { root_page_, col_num, col_tot_len, ... } // 直写，不经缓冲池
page 1 : IX_LEAF_HEADER_PAGE（叶链哨兵）
page 2+: IxNodeHandle { IxPageHdr{parent, num_key, is_leaf, prev_leaf, next_leaf}
                        + keys[] + rids[] }
```

7.2 核心操作

操作	实现	复杂度
find_leaf_page	从根按 internal_lookup 逐层下降	O(log N)
get_value	定位叶 + leaf_lookup 二分	O(log N)
insert_entry	定位叶 → 插入 → 满则 split + insert_into_parent	O(log N)
delete_entry	定位叶 → erase_pair	O(log N)
lower_bound / upper_bound	区间扫描起点定位	O(log N)

7.3 实现边界与设计取舍

当前 coalesce_or_redistribute / redistribute / coalesce / adjust_root 均返回 false，即未启用节点重分配、合并与根调整路径；erase_leaf 当前也未进入有效调用路径。因此，现有实现中的叶节点删除后不会执行合并。

该实现是针对当前赛题范围作出的工程取舍：节点合并会显著增加并发与结构维护复杂度，而当前测试与负载重点不依赖删除后的树收缩能力。在已覆盖的验证范围内，该取舍未影响功能结果，但会降低删除密集型工作负载下的空间利用率，因此不应视为通用数据库场景下的完整 B+ 树删除实现。

7.4 并发控制现状

root_latch_ 是每索引一把互斥锁，读者与写者共用，且在整个调用期间持有（ix_index_handle.cpp 的 insert_entry / delete_entry / get_value / lower_bound / upper_bound 入口处）。

这是当前已知的主要并发瓶颈：NewOrder 每笔事务要插 10 个 order_line 索引项，全部在一棵 1,500 万行的树上串行。改进方向是 B-link 树 + latch coupling（参考 PostgreSQL src/backend/access/nbtree/README），但受 §11 的环形叶链约束，因此，引入其他 B+ 树并发算法时必须针对本项目的叶链结构重新验证其不变量与终止条件。

8. 事务与并发控制

8.1 MVCC 架构：单一真相源 RowVersion

每个 (tab_name, rid) 维护一份 RowVersion：

```
struct CommittedVersion {
    timestamp_t commit_ts;
    bool is_deleted;                // 墓碑也保留 pre-delete 数据，给老快照读
    std::shared_ptr<RmRecord> data;
};

struct PendingWrite {
    enum class Op { NONE, INSERT, UPDATE, DELETE_OP } op;
    txn_id_t writer;                // INVALID_TXN_ID 表示无 pending
    std::shared_ptr<RmRecord> new_image; // INSERT/UPDATE 的新值
    std::shared_ptr<RmRecord> pre_image; // pending 之前 chain[0] 的快照
    bool pre_existed;                // 之前 chain 是否已有已提交版本
};

struct RowVersion {
    std::vector<CommittedVersion> chain; // [0] = 最新已提交，按 commit_ts 由新到旧
    PendingWrite pending;
};
```

关键不变量：

1. 所有可见性判断基于 RowVersion，不读堆。堆只用于扫描算子枚举 rid 以及索引指向的物理位置。
2. 写操作只更新 RowVersion.pending，不改 chain。
3. commit 时把 pending 转成 chain[0]。
4. abort 时丢弃 pending；INSERT 需由调用方撤销堆/索引（chain 为空）。
5. 由此，commit / abort 的版本状态变更集中在 RowVersion 中完成，减少 heap、元数据与 undo 状态之间出现不同步中间态的风险。

8.2 版本存储分片

版本存储按 (tab_name, page_no) 分成 **64 个分片**（2 的幂，取模用位与）：

```
static constexpr size_t VERSION_SHARD_NUM = 64;
struct VersionShard {
    std::shared_mutex mutex_;
    std::unordered_map<std::string,
        std::unordered_map<page_id_t, std::shared_ptr<PageRows>>> pages_;
};
struct PageRows {
    std::shared_mutex mutex_;
    std::unordered_map<int, RowVersion> rows_;    // key: slot_no
};
```

原实现是单一全局 `shared_mutex` 保护一张大表：`get_or_create_page_rows` 每次写都要独占它，而它是所有 INSERT/UPDATE/DELETE 的必经路径，32 客户端下所有写在此排队；读路径每扫一行取一次共享锁，同一共享状态会在多个 CPU 核之间产生高频缓存一致性竞争。

该设计采用分片哈希与分区锁的思路，将原先的单一全局锁拆分为多个独立锁域。分片数量需要在并发度与全量遍历成本之间权衡：分片越多，写竞争越低，但 GC 等需要遍历全部分片的操作成本越高，当前取 64。

分片函数必须只依赖 (`tab_name`, `page_no`)，无随机、无状态、无时间依赖。若两个线程把同一页算到不同分片，会各自建一份 `PageRows`，同一行可能生成两份独立版本状态，从而破坏可见性判断的一致性。

8.3 快照隔离 (SI)

- `begin()` 取快照时间戳 `start_ts`
- 可见性：沿 `chain` 找第一个 `commit_ts <= start_ts` 的版本；`is_deleted` 则返回 `nullptr`
- 自己的 `pending` 优先可见
- 写写冲突：first-committer-wins，冲突方抛 `TransactionAbortException`

8.4 提交时间戳与版本发布时序

该时序直接决定快照可见性与提交发布的一致性。

`commit_ts` 在 `commit()` 顶部分配，但版本要到 `FinalizeCommit` 才进入 `chain`。如果 `begin()` 读 `last_commit_ts_`，就可能拿到一个「已分配 `commit_ts` 但版本尚未落链」的快照，于是同一事务的两次读会不一致。

因此 `begin()` 读的是**已发布水位** `published_ts_locked()`，不是 `last_commit_ts_`。

因此，FinalizeCommit 的相对位置不能后移到 group_commit 之后。否则，版本发布时间与持久化同步之间的竞态窗口会被显著放大；项目实测表明该改动会破坏 Payment 场景下的 FLOAT32 0-ULP 校验。

8.5 可串行化快照隔离（SSI）

在 SI 基础上跟踪 rw 反依赖，检测「危险结构」 $T_{in} \rightarrow_{rw} T_{pivot} \rightarrow_{rw} T_{out}$ 。

API: SsiRegisterBegin / SsiRecordRead / SsiRecordWrite / SsiRecordPredicate /
SsiCheckInvisibleWriters / SsiMarkCommitted / SsiMarkAborted / SsiGarbageCollect。

危险结构判定（ssi_is_dangerous_pattern）：

```
if (tin.txn_id == tout.txn_id) return true;           // 自环
if (tout 已提交 && tin 未提交未回滚) return true;    // Tout 必然先提交
if (两者都已提交) return tout.commit_ts < tin.commit_ts; // 比较提交序
```

赛题额外约定：形成危险结构时必须由**当前语句立即返回** TRANSACTION_ABORT

并回滚当前事务，不得推迟到 COMMIT 阶段，也不得改选其他回滚对象（victim）。这比「任意正确的可串行化

实现均可在 COMMIT 时裁决 victim」的通用表述更严格。

不可见写者处理：扫描遇到对本事务不可见的行时，对「内容满足本扫描谓词」的不可见写者建立 $W \rightarrow me$ 反依赖（不计入 rid_reads）。当前可见旧值不在结果中、但不可见新值可能被更新进查询范围的情况同样要建边。

8.6 水位线与垃圾回收

Watermark 维护最小活跃读时间戳：

```
void AddTxn(read_ts) { current_reads_[read_ts]++; watermark_ = 最小键; }
void RemoveTxn(read_ts) { 计数减一，清零则删；空表时 watermark_ = commit_ts_; }
timestamp_t GetWatermark() { return current_reads_.empty() ? commit_ts_ : 最小键; }
```

版本存储 GC：每 GC_COMMIT_INTERVAL = 10000 次提交跑一次 sweep，把已 settled（commit_ts < watermark、无 pending、非墓碑）的 RowVersion **整条删除**，使版本存储规模为 O(活跃工作集) 而非 O(总行数)。

现有实测表明，若不执行该回收：海量灌库（每条 insert 一个自动提交事务）会让版本存储持续增长，并可能在 normal 阶段触发 OOM，导致服务端进程被系统终止。

同理，`reap_transaction` 用于回收 `txn_map` 中已结束事务，避免事务对象持续累积。由调用方在**同连接的下一条语句**开始时对上一条已结束的事务调用——此刻该指针不再被任何线程解引用（提交后 `pending` 已清、SSI 用 `txn_id` 而非指针），无 `use-after-free`。

8.7 锁管理器的现状

`LockManager` 类保留在代码中，但当前数据访问路径未使用该组件。SI/SSI 主要通过 MVCC、版本可见性与冲突检测实现，不采用两阶段封锁作为主并发控制机制。`lock_manager.h` 中的条件变量也未进入当前 `no-wait` 数据路径。

9. 日志与故障恢复

9.1 WAL 格式

日志记录类型：`begin / commit / ABORT / INSERT / UPDATE / DELETE / StaticCheckpoint`。每条记录含 `log_tot_len / log_type / lsn / log_tid / prev_lsn`，数据记录另带表名、`rid` 与前后像。

`LogManager` 维护 4 MiB 日志缓冲（`1024 * PAGE_SIZE`）：

```
lsn_t add_log_to_buffer(LogRecord*);    // 持 latch_，缓冲满则先刷盘
void flush_log_to_disk();               // write_log + sync_log(fdatasync)
```

9.2 提交持久化契约

官方对 `COMMIT` 有可核验的审计要求：**`COMMIT` 成功响应之前，对应 WAL commit record 必须已被同一 ACK 窗口内的稳定化操作覆盖。**

实现上 `commit()` 写完 `CommitLogRecord` 后直接 `flush_log_to_disk()`，其中 `write_log` 追加正字节、`sync_log` 调 `fdatasync`。允许 group commit，但不得「先 ACK 多个再补一次同步」。

合法 WAL 命名空间：`db.log`、根目录下非空后缀的 `db.log.*`、`wal/**`。

9.3 恢复三阶段

analyze() → redo() → undo() → rebuild_after_recovery() → reset_runtime_state()

analyze (pass 1, 流式)

- 确定扫描起点：存在重启文件 db.ckpt 则从检查点偏移开始， from_checkpoint_ = true
- 统计 max_lsn_ / max_tid_
- 算出崩溃时未提交的事务集合及其 INSERT 清单

在途事务表只保留 INSERT (commit / abort 即丢弃)，因此内存占用与日志总量无关。

日志扫描 scan_log 用 8 MiB 滚动缓冲分块读：跨块的半条记录搬到缓冲头部后续读补齐，单条超过缓冲容量时按需扩容。内层循环把缓冲里能解析的完整记录全部吃掉，搬移/读盘只发生在外层——否则每条记录 memmove 一次 8 MiB 缓冲，回放退化成 O(n·缓冲)。

另有内存治理：每处理 4 M 条记录调一次 malloc_trim(0)。每条记录的瞬时小对象虽然即放即收，但 glibc 高水位不回缩，实测 2,400 万条日志会让 RSS 爬升约 2.2 GiB。

redo (pass 2)

先 ensure_pages_on_disk() 把各表数据文件物理扩展到 header.num_pages，保证按 rid 重放时目标页一定存在。然后顺序重放 INSERT/UPDATE/DELETE。

恢复扫描会跳过非法 RID (page_no < 0 或 slot_no < 0) 对应的日志记录；否则按该 RID 重放时可能触发 fetch_page(-1) 异常并中断整个恢复流程。

undo

逆序撤销未提交事务的 INSERT (delete_record)。MVCC 下未提交的 UPDATE/DELETE 从未写堆，无需处理。若某 rid 已被已提交事务接管则跳过。

rebuild_after_recovery

路径	空闲链	索引
无检查点	全表 recover_free_list()	全部表从堆全量重建
有检查点	不重算 (检查点已一致落盘，redo/undo 用的 insert_record(rid) / delete_record 本身增量维护 bitmap、num_records 与空闲链)	只重建 touched_tabs_

索引恢复以堆文件为可信数据源，不采用按 RID 对现有索引结构进行增量修补：运行期 B+ 树分裂/页分配把头页与数据页以不同步的方式落盘，崩溃后树结构可能已不一致，往里 insert_entry 会触发 find_child 断言（实测）。因此恢复阶段以堆文件作为索引重建的数据源。

MVCC 版本不在此全表播种——由运行期 VisibleRecord / ProbeRow 惰性补种，使恢复时间正比于实际访问量而非全量。

9.4 静态检查点

```
void create_static_checkpoint() {
    lock_guard guard(checkpoint_mutex);
    txn_manager->BeginCheckpointQuiesce();           // (1) 静止化
    try {
        txn_manager->ApplyLogicalDeletesToHeap();    // (2) 兑现逻辑删除
        log_manager->flush_log_to_disk();            // (3) 刷日志缓冲
        ckpt_offset = get_file_size(LOG_FILE_NAME);
        log_manager->add_log_to_buffer(&ckpt_rec);    // 写检查点记录
        log_manager->flush_log_to_disk();
        for (堆文件) { flush_file_hdr(); flush_all_pages(fd); } // (4) 刷脏页+文件头
        for (索引) { flush_index(ih); }
        写 db.ckpt (检查点记录地址);                // (5) 重启文件
    } catch (...) { EndCheckpointQuiesce(); throw; }
    txn_manager->EndCheckpointQuiesce();
}
```

以下两项是保证静态检查点恢复正确性的关键条件：

(2) ApplyLogicalDeletesToHeap：MVCC 的 DELETE 不动堆，物理删除只发生在恢复 redo 里。而检查点路径的 redo 从检查点偏移起扫，检查点之前的 DELETE 再也不会被重放。若缺少该步骤，检查点之前已提交删除的记录可能在崩溃恢复后重新出现。该步骤必须在静止化期间执行：此刻无活跃事务，没有读者还需要这些旧版本；若在事务运行时清堆 bitmap，老快照的顺序扫描会直接跳过该行而不经版本链。

(4) 两类 file header：RmFileHdr 与 IxFileHdr（含 root_page_）都由 disk_manager->write_page 直写、不经缓冲池，flush_all_pages 碰不到它们。而检查点路径的恢复对未被触及的表既不重建索引，也不重新计算空闲链，因此依赖磁盘上的元数据状态。若文件头未持久化，崩溃后索引句柄可能读取到陈旧的 root_page_，从而导致索引结构严重不完整。实测（零负载、发一次

检查点再 SIGKILL)： order_line 索引 15,002,806 → **84** 条， stock 5,000,000 → **127** 条。此前从未暴露，是因为无检查点时恢复总会从堆全量重建所有索引。

静止化的超时保护： BeginCheckpointQuiesce 等待 active_txns_ == 0 时设 10 秒上限，避免连接长期持有未提交事务时使检查点线程无限期等待。EndCheckpointQuiesce 必须在异常路径上也执行，否则后续所有 begin() 会被永久阻塞（begin() 对 checkpoint_in_progress_ 的等待是无超时的）。

10. 关键设计决策与权衡

#	决策	设计收益与代价
1	MVCC 以 RowVersion 作为版本状态中心	收益：commit / abort 的版本状态集中处理，降低多状态源不同步风险。 代价：版本需要驻留内存，必须配套 GC。
2	只读扫描不主动补种版本	收益：避免全表扫描带来 O(表规模) 的额外版本内存。 代价：需要满足第 5.3 节给出的可见性前提。
3	版本存储采用 64 分片	收益：降低全局写锁竞争。 代价：GC 等全量遍历操作成本上升，且分片函数必须保持确定性。
4	B+ 树当前不执行节点合并	收益： 降低当前赛题范围内删除路径的结构维护复杂度。 代价：删除密集型负载下空间利用率下降，通用性受限。
5	B+ 树采用每索引单互斥锁并覆盖完整操作	收益：实现简单，便于维持树结构一致性。 代价：高并发写入时形成明显串行瓶颈。
6	SERIALIZABLE 强制 SeqScan	收益：SSI 读集与谓词登记保持既定表粒度语义。 代价：SERIALIZABLE 下无法利用索引扫描加速。
7	skip scan 仅支持单个未绑定 INT 前导列	收益：后继值计算与边界控制较简单。 代价：适用范围有限，并设置 4096 次探测保护阈值。

#	决策	设计收益与代价
8	MIN 下推暂不扩展到 MAX	收益：复用现有正向索引扫描， 无需引入环形叶链上的反向终止逻辑。 代价：MAX 仍需扫描完整候选区间。
9	每连接一线程	收益：连接处理模型简单，适配赛题固定 32 客户端的并发规模。 代价： 连接数显著扩大时线程调度与栈内存开销会上升。
10	检查点采用全局静止化	收益：检查点落盘期间无并发事务修改数据， 便于保证一致磁盘快照。 代价：静止期间新事务需要等待，并设置 10 s 等待上限。

11. 关键实现不变量与风险约束

以下条目描述当前实现正确性依赖的重要不变量与边界条件。修改相关模块时，应保持这些约束成立，或在变更设计中提供等价的正确性保证。

11.1 B+ 树叶链是环形的

哨兵页 `IX_LEAF_HEADER_PAGE = 1` 被创建为 `is_leaf = true`，并且双向指向第一个叶子。因此：

`is_leaf_page()` 不能用来判断扫描结束。游标走过最后一个叶子会绕回来，无限循环。

`IxScan` 通过以下机制共同保证扫描能够正确终止；修改时应保持等价的终止语义：

- 哨兵页视为终点
- `ended_` 标志不可逆
- 页内 `slot_no >= end_.slot_no` 单调判定
- `copy_current_key` 的边界保护
- 叶子步进次数上限

若引入默认叶链以 `NULL` 结尾的 B+ 树算法，需要针对本项目的环形叶链重新验证遍历边界与终止条件。

11.2 叶子页号不代表扫描位置

页号反映物理分配顺序而非键序，因此不能使用 `page_no >` 之类的页号比较判断扫描进度。

11.3 两个文件头在缓冲池之外

`RmFileHdr` 与 `IxFileHdr` 由 `disk_manager->write_page` 直接写入，不经过缓冲池，因此 `flush_all_pages` 不会覆盖这两类文件头。检查点必须通过 `flush_file_hdr()` / `flush_index()` 显式持久化。

11.4 MVCC DELETE 不动堆

墓碑只存在于内存版本链。物理删除发生在恢复重放已提交 `DELETE` 时，以及检查点的 `ApplyLogicalDeletesToHeap()` 阶段。若检查点前的逻辑删除未被兑现到堆文件，崩溃恢复后可能重新暴露已删除记录。

11.5 可见性不能跑在发布之前

`begin()` 必须读取已发布水位而非 `last_commit_ts_`，同时应保持 `FinalizeCommit` 与版本发布顺序不被后移。

详见 §8.4。

11.6 `fetch_page` 会返回 `nullptr`

分片耗尽时返回空指针，不进行阻塞等待或自旋，而多数调用方直接解引用。新增调用点时注意。

11.7 `update_page` 在持分片 latch 时做 I/O

淘汰脏页会在持有分片 latch 的状态下执行 `write_page`，并可能触发 `flush_log_cb_`。调整锁粒度或加锁顺序前，需要验证不会形成锁顺序反转或新的 I/O 临界区竞争。

11.8 `// TODO`：不是可靠的未实现标记

框架中部分保留 `// TODO`：注释的函数已经包含有效实现，因此判断实现状态时应以函数体实际行为为准，不能仅依据历史注释。

12. 性能分析与优化结果

12.1 代表性官方评测指标（main @ 0e544e4）

指标	值
NewOrder/min（三窗口中位数）	5735.6
abort 率	8.10%
p50 延迟	11.827 ms
p99 延迟	180.645 ms
CPU 占用	1458%（16 核的 91.1%）
装载耗时	65.67 s
崩溃后恢复 + 校验	74.96 s
峰值 RSS	1.34 GiB
峰值库大小	5.41 GiB

与早期记录的 174.4 NewOrder/min 基线相比，该评测结果对应约 **32.9×** 的吞吐提升。

12.2 主要性能收益来源

从现有优化记录看，R036 的索引跳跃扫描是整体吞吐提升的主要贡献项。

TPC-C Delivery 中的 `select sum(ol_amount) from order_line where ol_d_id=? and ol_o_id=?` 没有绑定索引首列 `ol_w_id`，传统最左前缀匹配无法使用现有复合索引，因而退化为约 1,500 万行的全范围扫描。Delivery 在事务混合中的比例为 4%，但每笔事务需要处理 10 个地区，该查询会重复执行 10 次。其高成本扫描不仅增加 Delivery 自身延迟，也会显著占用缓冲池与内存带宽，进而降低其他事务类型（包括 NewOrder）的有效吞吐。

R036 之后的若干轮次主要用于补强正确性、持久性、恢复一致性以及性能结果的可验证性，其主要目标并非直接提升吞吐：

轮次	内容
R037	崩溃后 YTD 更新链一致性
R038	提交发布水位 (§8.4)

轮次	内容
R039	检查点元数据持久化与删除安全恢复 (§9.4)
R041	MIN 沿索引序下推
R042	版本链剪枝
R043	Delivery 排队分析

12.3 性能优化方法与工程实践

性能优化过程采用“规范约束 → 瓶颈定位 → 单一假设 → 最小修改 → 正确性门禁 → 性能复测”的迭代方式。

现有优化记录归纳出以下工程原则：

- 以官方规范和真实评测负载为优化边界。** 在建立优化假设前先核对 docs/ 中的赛题与一致性规则，避免在官方验证范围之外投入主要优化资源。
- 性能样本必须来自真实或等价负载。** 调试器中观察到的热点只有在对应 SQL 与官方负载形态一致时，才能用于推导性能结论；自构造查询只能用于局部诊断，不能直接替代官方负载。
- 每轮尽量验证一个可证伪假设。** 将多个无关改动打包会降低回归失败后的可定位性，因此每轮应控制修改范围，并保留可回滚、可比较的记录。
- 正确性门禁优先于性能结论。** 若必过门禁失败，则该轮性能结果不作为有效优化结论，也不用于更新性能基线。
- 测试脚本需要验证诊断能力。** 门禁不仅要在正常构建上通过，还应确认其能在已知缺陷构建上稳定失败，从而证明该测试确实覆盖目标故障模式。

12.4 当前主要性能瓶颈

优先级	问题	位置
P1	索引写路径全局锁。 root_latch_ 每索引一把、全程持有、读写共用。NewOrder 每笔插 10 个 order_line 索引项，全在一棵 1,500 万行树上串行	ix_index_handle.cpp
P2	堆插入锁粒度。 heap_latch_ 每表一把，覆盖 INSERT 与 DELETE。Delivery 删 new_orders 与 NewOrder 插	rm_file_handle.cpp

优先级	问题	位置
	new_orders 互斥	
P3	IndexJoinExecutor 不累加外侧 rows_out_，EXPLAIN ANALYZE 里 INLJ 外侧恒显示 rows=0（仅影响可读性）	executor_index_join.h

13. 测试与验证体系

13.1 分层门禁

对于触及存储、索引、恢复或事务模块的变更，需通过以下分层门禁后再采纳性能结果：

门禁	判定标准
regress.sh ×2 + diff	28 个 scripts/sql/*.sql，与基线逐字节一致
full_check.py	官方 4016 条查询校验，无 ERROR
index_integrity.py	8/8 —— 顺序扫描计数 == 索引扫描计数
partition_closure.py	4/4 delta=0 —— 唯一覆盖带 stop key 的范围扫描的门禁
freelist_test.py	6/6 —— 在 恢复之后 插入；空闲链损坏只在这里暴露
durability_test.py	通过 SIGKILL 进行故障注入：已 ACK 的提交存活，未提交的回滚
durability_bigtxn.py	4.4 MiB 事务跨越 4 MiB 日志缓冲
payment_f32_snapshot.py	0 处位级不一致 —— 快照稳定性 + FLOAT32 0-ULP
crash_checkpoint_consistency.py	检查点路径崩溃恢复 ：堆/索引一致、YTD 链、已 ACK 的删除保持删除
skipscan_equiv.py	127 项差分校验（改动 planner 或索引扫描时必跑）
min_pushdown_equiv.py	MIN 下推与全区间扫描结果等价
unit_test	6/6（改动存储层时才有意义）

13.2 测试门禁的诊断有效性

除在正确构建上通过外，关键门禁还需要在已知缺陷构建上能够稳定失败，以验证其诊断有效性。当前已确认具备该能力的测试包括：

门禁	在哪个构建上失败	表现
payment_f32_snapshot.py	514c1ef	283/15132 处位级不一致
ckpt_isolate.py	8acd920	order_line 索引 15,002,806 → 84 条
crash_checkpoint_consistency.py	8acd920	索引损坏 + 400/400 已 ACK 的删除复活

作为反例，ytd_update_chain.py 与 ytd_chain_relational.py 在已知存在官方崩溃后一致性问题的 8acd920 构建上仍可通过，因此不能诊断该类故障，现已归档并不再作为对应正确性结论的依据。

13.3 性能测量规范

tmp/record/gates/ab.sh <label> <bin> [seconds] 按官方事务形态驱动 32 个客户端，并采集 3 个测量窗口。

- **负载形态必须与官方保持一致。**本地测试需设置 `RMDB_DELIVERY_SUM=official`；否则驱动会在 Delivery 的 `sum(ol_amount)` 查询中额外加入 `ol_w_id`，从而改变查询计划与代价，所得结果不具备与官方评测直接比较的意义（R036 之前该差异表现为本地 12.5 ms，而官方形态为 5029 ms）。
- **不同硬件与窗口配置下的绝对值不直接比较：**本地环境为 20 核、40 s 窗口，评测环境为 12~16 核、150 s 窗口。
- **性能结论需要考虑窗口波动：**现有记录中同组 3 个窗口的结果波动最高可达 2.2×；当组间差异落入该波动范围时，不应仅据此认定优化具有稳定收益。

13.4 已知失效的框架测试脚本

scripts/run_sql_test.sh 与 run_all_tests.sh 仍依赖服务端已经不再生成的 output.txt（g_output_enabled 在 rmdb.cpp 中恒为 false）。脚本在出现 "No such file" 后仍可能以 exit 0 结束，因此会造成测试状态误判。当前回归验证统一使用 tmp/record/gates/regress.sh。

14. 已知限制与后续方向

14.1 功能限制

- **无 NULL 支持。** Value 结构体没有 is_null 字段，ColMeta / RmRecord 亦然。协议层的 present=0 是保留编码，正式 TPC-C 负载不写入也不绑定 SQL NULL（未配送的整数列用 0，字符时间戳用空字符串）。
- **无外连接。** ast.h 里 JoinType{INNER,LEFT,RIGHT,FULL} 与 JoinExpr 已定义但全仓库无第二处引用，是框架预留的扩展点。做 LEFT JOIN 的前置条件是先实现 NULL。
- **无 OR 谓词。** whereClause 是纯左递归 AND 列表，不支持析取。
- **B+ 树删除后不执行节点合并**（见 §7.3）。该限制属于当前赛题范围下的工程取舍。

14.2 后续性能优化方向

结合当前热点与实现复杂度，后续优化方向按预期优先级排列如下：

1. **B+ 树并发：**B-link 树 + latch coupling / 乐观锁耦合。参考 PostgreSQL src/backend/access/nbtree/README、BusTub 的并发 B+ 树。实施前需先处理 §11.1 所述的环形叶链约束。
2. **堆插入锁粒度：**参考 PostgreSQL 的 Free Space Map 与 RelationGetBufferForTuple。
3. **WAL 插入锁：**PostgreSQL 用一组 WALInsertLock 把插入与刷盘分离；MySQL 8.0 把单一 log_sys mutex 换成线程流水线。
4. **并行扫描：**DuckDB/Umbra 的 morsel-driven parallelism。

14.3 许可证与第三方参考约束

项目代码与第三方资料的使用应同时满足仓库许可证、第三方项目许可证以及赛事规则。

当前文档记录 RMDB 采用木兰宽松许可证第 2 版（Mulan PSL v2）；其他第三方项目的具体许可证边界应以各自仓库中的 LICENSE 文件为准。

- 对 MySQL/InnoDB 等与本项目许可证条件不同的实现，仅用于理解公开设计思想，不直接复制其源代码。
- 对 PostgreSQL、BusTub、DuckDB 等公开项目，可将其论文、设计文档或公开实现作为原理参考，但项目代码应结合本系统结构独立实现。
- 第三方资料的参考关系、关键设计依据及 AI 辅助使用情况应按比赛要求保持可追溯记录，不以“消除痕迹”作为文档或代码处理目标。
- 当前项目中的相关参考记录位于对应优化轮次的 record.md 与 skills/ai使用.md。

附录 A：目录结构

src/	
└─ rmdb.cpp	服务端主程序：监听、连接处理、协议分发、检查点
└─ portal.h	Plan 树 → Executor 树
└─ common/	
└─ config.h	页大小、缓冲池容量与分片数、日志缓冲大小
└─ common.h	Value / TabCol / Condition / AggType
└─ context.h	执行上下文（事务、日志、锁、结果出口）
└─ wire_protocol.h	Wire Protocol v3 编解码（权威定义）
└─ parser/	flex + bison → AST
└─ analyze/	语义分析：AST → Query
└─ optimizer/	
└─ plan.h	PlanTag 与各 *Plan 类
└─ planner.cpp	索引选择、连接算法、skip scan / MIN 下推授权
└─ execution/	16 个火山模型算子 + EXPLAIN 打印
└─ system/	数据库目录：DbMeta / TabMeta / ColMeta / IndexMeta
└─ record/	堆文件：位图页、空闲链、扫描游标
└─ index/	B+ 树
└─ storage/	缓冲池（分片）+ 磁盘管理
└─ replacer/	LRU
└─ transaction/	MVCC + SI/SSI + 水位线
└─ recovery/	WAL 与故障恢复

附录 B：核心常量

常量	值	位置
PAGE_SIZE	4096	common/config.h
BUFFER_POOL_SIZE	262,144 页 (1 GiB)	common/config.h
BUFFER_POOL_SHARD_NUM	16	common/config.h
日志缓冲	1024 × PAGE_SIZE (4 MiB)	common/config.h
VERSION_SHARD_NUM	64	transaction/transaction_manager.h
GC_COMMIT_INTERVAL	10,000	transaction/transaction_manager.h
kMaxSkipProbes	4096	execution/executor_index_scan.h

常量	值	位置
IX_LEAF_HEADER_PAGE	1	index/ix_defs.h
监听端口	8765	rmdb.cpp
帧 payload 上限	1 MiB	common/wire_protocol.h
诊断文本上限	64 KiB	common/wire_protocol.h

附录 C：构建与运行

```
# 构建（Release，与官方 -O2 -DNDEBUG 一致）
cmake --build build-rel --target rmdb unit_test -j"${nproc}"

# 运行：chdir 进 <db_name>/，监听 TCP 8765
./build-rel/bin/rmdb <db_name>

# 故障恢复测试：模拟服务端被强制终止
pkill -9 -x rmdb          # 仅用于故障注入；避免使用过宽的 -f 模式匹配
```

四个构建目录不可互换：

目录	类型	用途
build-rel/	Release	日常工作构建
build-dbg/	RelWithDebInfo	gdb / 性能剖析
build-base/ 、 build/	Release	A/B 对照的基线二进制，保留为对照环境，不参与日常重建

运行就绪判定：服务端会在恢复完成前绑定监听端口，因此 `connect()` 成功并不能证明数据库已经可用。

当前记录中崩溃后启动的重建阶段约需 11 秒，具体耗时受数据规模与运行环境影响。应以 Wire Protocol 成功执行 `show tables;` 作为服务端完成恢复并进入可服务状态的判定依据。